

Forging Tree-Ring: A Reproduction Study of Semantic Watermark Forgery

Saifur Rahman Tamim¹, Md Taslimul Hasan Toufique¹, and A.M. Tayeful Islam¹

Project Group 7*

¹Department of Computer Science and Engineering

Northern University Bangladesh, Dhaka, Bangladesh

tamim_41230201087@nub.ac.bd, ripperthemodder@gmail.com

tayef_nubcse@nub.ac.bd

Abstract

Semantic watermarking schemes such as Tree-Ring hide a detectable pattern in the initial noise latent of a diffusion model. Recent work shows these watermarks are not only removable but forgeable: an attacker who never sees the watermarking key can still produce images the genuine detector accepts. We reproduce the Reprompt forgery attack of Müller et al. against Tree-Ring on Stable Diffusion XL, using the authors' released code, on free-tier dual T4 GPUs with 14.6 GB of usable memory per device, substantially less per-GPU memory than the A40 hardware used in the original study.

The attack reproduces. Over six trials of three arms we detect genuine images 6/6, clean images 0/6, and forged images 5/6, at 332.0 s per attack. Three further results came out of running it under constraint. The released detector computes a non-central χ^2 statistic and hands back only its CDF, so we recovered the discarded statistic; our recovery matches the released detector exactly, and two natural scores built from it separate the forged arm from the clean null at AUC 0.861 and 0.972 on the same eighteen observations. Running SDXL in half precision requires patching the pipeline's direct autoencoder calls, and a controlled probe confirms the patched path leaves the detector statistic unchanged. Finally, we report a prediction we made from reading the detector source that our measurements then contradicted. The notebook, the pinned fork and every measurement artifact are released with the paper.

1. Introduction

Text-to-image diffusion models are good enough now that asking whether a picture was machine-generated has stopped being an academic question. Watermarking is one prominent answer. Among the watermarking families, semantic schemes are notable because they add no post-hoc pixel-space perturbation: they encode a signal in the initial noise latent the image is generated from, so the mark is a property of the generative trajectory rather than something applied to a finished image. Tree-Ring [15] is the canonical example, and it survives the compression, cropping and blurring that defeat pixel-domain marks.

Surviving distortion is not the only thing a provenance mechanism has to do. A watermark that resists every attack on its

robustness is still useless as evidence of origin if someone can manufacture it. Müller et al. [10] showed exactly that against Tree-Ring and Gaussian Shading [17]. The ring pattern survives DDIM inversion carried out in a *different* diffusion model, so an attacker who owns one watermarked reference image can invert it in a proxy model and regenerate arbitrary new images the genuine detector will accept. No key, no gradients, no privileged access to the target. Figure 1 sketches the whole thing.

We set out to reproduce that attack independently. Not because we doubted it, but because we wanted a verified baseline on hardware we control before building anything on top of it. Reproduction turns out to be the right first step more often than the literature admits: a longitudinal study of roughly 750 machine-learning security papers found that 60% shipped no code, and that 56% of the artifacts which did exist would not run [11].

Our constraint was compute. Müller et al. evaluated on NVIDIA A40 hardware, distributing their experimental campaign over eight such GPUs; we had a free-tier notebook service offering two NVIDIA T4 accelerators with 14.6 GB of usable memory each. The released code assumes full precision throughout and has no half-precision path. Most of what we report below came out of working inside that limit, because it forced us to instrument parts of the pipeline that a bigger machine would have let us ignore.

Contributions.

- A verified reproduction of Reprompt against Tree-Ring on SDXL under a fully pinned software and provenance configuration, with genuine, clean and forged detections all landing inside the originally reported distributions (Section 6).
- Recovery of the detector's discarded non-central χ^2 statistic, matching the released detector exactly, and the observation that two scores derived from it give materially different separation on identical samples (Section 5).
- A mixed-precision compatibility fix for SDXL's direct autoencoder calls, with a controlled probe showing the patched path returns the same detector output as full precision on the tested sample (Section 4.4).
- A four-gate feasibility protocol covering memory, cost, statistic recoverability and hypothesis separation, re-

*This paper serves as the final project report for CSE 4383: Image Processing and Computer Vision Lab Work, Northern University Bangladesh.

Defender (holds key k)

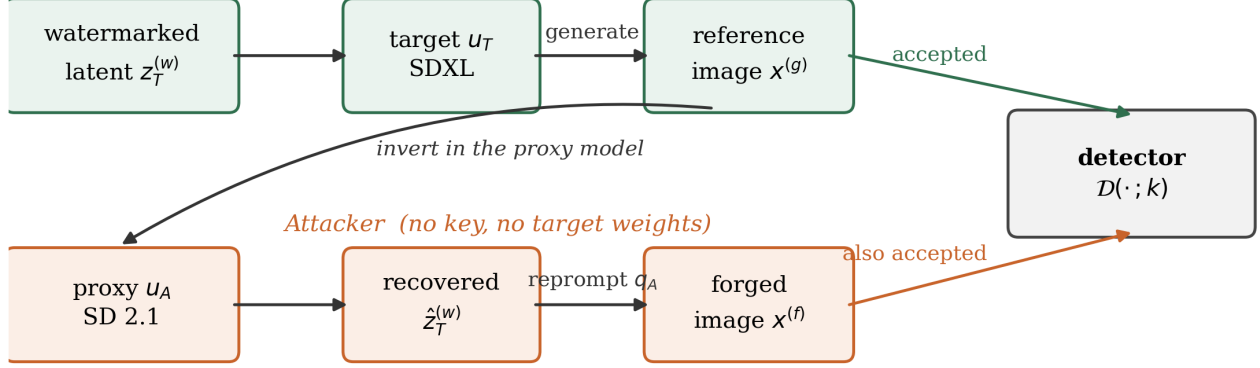


Figure 1: The Reprompt forgery attack. The defender generates from a key-derived latent (top). The attacker takes one reference image, inverts it in a proxy model that has never seen the key, and regenerates an unrelated image from the recovered latent (bottom). Both images are accepted by the same detector.

ported with measured outcomes and including one prediction our data refuted (Sections 6 and 7).

- A battery of post-hoc statistical tests on the gate data — bootstrap confidence intervals, pairwise Mann-Whitney tests and effect sizes — so the recovered separations carry a quantified uncertainty (Section 6).

2. Background and Related Work

2.1. Semantic watermarking

Frequency-domain watermarking predates diffusion models by decades [4], and the learned pixel-space variants that followed, such as HiDDeN [21], embed a message with a jointly trained encoder-decoder pair. A third family fine-tunes the generator itself so that its outputs carry a signature, as in the Stable Signature [5]. Semantic schemes take a fourth route.

A latent diffusion model [7, 13] builds an image by repeatedly denoising an initial Gaussian latent z_T . Tree-Ring [15] watermarks the model by overwriting part of z_T in the Fourier domain with a fixed, key-derived pattern: concentric rings placed in one channel of the shifted spectrum. Since the pattern goes in before generation rather than after, it is carried through the entire sampling trajectory and is not localised anywhere in the output.

Detection runs the process backwards. The detector inverts the sampler with DDIM [14] to estimate the latent the image would have come from, applies the same frequency-domain mask, and compares the recovered coefficients against the key pattern. With f the recovered masked coefficients, t the key pattern and σ the standard deviation of f , the released implementation computes

$$x = \sum_i \left(\frac{f_i - t_i}{\sigma} \right)^2, \quad \lambda = \sum_i \frac{t_i^2}{\sigma^2}, \quad (1)$$

and reports $p = F_{\chi^2/2}(x; \text{df}, \lambda)$. A detection is declared when $p < \tau$; for SDXL [12] at a 1% false-positive rate, $\tau = 0.0261$. One detail matters later: detection works on an *estimate* of z_T ,

since DDIM inversion is only approximately invertible. Every detection is already one inference removed from the quantity the scheme embeds.

Ci et al. [3] revisited the scheme and found that part of its robustness comes not from pattern matching but from a distribution shift the watermarking process introduces incidentally. They also showed that Tree-Ring handles multiple distinct keys poorly, where that shift gives no help, and proposed RingID to address it. Both observations bear on our own single-key limitation (Section 8).

2.2. Removal versus forgery

Most of the attack literature goes after removal, destroying the mark while keeping the image usable. Zhao et al. [18] give the strongest general result in that direction: adding noise in latent space and regenerating provably removes pixel-level watermarks, though they observe that semantically-embedded marks resist the same attack, which is part of why semantic schemes drew attention in the first place. The WAVES benchmark [1] organised this into a stress-test suite spanning distortion, regeneration and adversarial attacks, and found Tree-Ring especially vulnerable to adversarial embedding and surrogate-detector attacks while holding up reasonably well against plain distortion.

Forgery runs the other way, and for provenance it is the more damaging direction. Removal causes false negatives: a genuine image stops being recognised. Forgery causes false positives: an image the marked model never produced gets attributed to it. A provenance claim depends entirely on the second error rate, and robustness results say nothing about it.

2.3. The Reprompt attack

Müller et al. [10] demonstrate that the ring pattern survives DDIM inversion in a model other than the one that generated the image. The attacker holds a watermarked reference $x^{(w)}$, inverts it in a proxy u_A to get $\hat{z}_T^{(w)}$, then generates a new image from that latent under whatever prompt they like. The output looks nothing like the reference and the genuine detector

Table 1: Original and reproduction environments.

	Müller et al.	This work
GPU	A40, 48 GB nominal	T4 $\times$ 2, 14.6 GB ea.
Precision	fp32	fp16 target / fp32 attacker
Target	SDXL	SDXL
Attacker	SD 2.1	SD 2.1
Resolution	512	512
Scheduler	DDIM	DDIM
Allocation	dedicated	30 GPU-h / week, shared

accepts it anyway.

The same paper introduces Imprint, an optimisation-based variant that learns a perturbation to a cover image’s latent so its inversion matches a target watermarked latent. Imprint buys finer control at roughly two orders of magnitude more cost per run. We reproduce Reprompt only, for reasons given in Section 8.

Later work has chased cheaper forgery. Zhu et al. [20] propose Plug-and-Plant, an optimisation-free attack that extracts and re-implants a watermark by regenerating the image through an off-the-shelf diffusion model, reporting up to 100% detectability across two dozen model-data-watermark combinations. Zhou et al. [19] approach the problem from the defensive side with MetaSeal, arguing that content-agnostic marks verified by a detector are structurally forgeable and that cryptographic, content-dependent signatures are the way out. Their framing is useful to us: it identifies the two properties that make a scheme forgeable in the way Tree-Ring is.

2.4. Reproduction as a research activity

Independent reproduction is an established genre. Olszewski et al. [11] measured its state across a decade of Tier-1 security venues, finding that 60% of papers shipped no code and that 56% of the artifacts which did exist would not run. They further report no statistically significant change in artifact *availability* after artifact evaluation committees were introduced, while noting that artifacts which passed evaluation worked at a higher rate than those that did not.

Reproductions that diverge are informative in their own right. Hwang et al. [8], reproducing Tree-Ring on the removal side, matched the original’s unattacked performance closely but got an AUC of 0.463 under rotation where the original reported 0.935. Rather than bury it, they traced the gap to rotation in pixel space not corresponding to rotation in latent space, and confirmed the effect held across channel placements, model versions and schedulers. We take the same posture below: where our measurements diverge, we say so.

3. Reproduction Setup

3.1. Hardware

Table 1 sets out the gap we were working across. The notebook service offers two accelerator types, and one of them is unusable: the P100 reports compute capability `sm_60`, which the installed PyTorch 2.10 build has dropped support for. That leaves the dual-T4 configuration. Neither T4 supports `bfloat16`,

which puts FLUX.1-dev out of reach even though the released code supports it. This is a hardware exclusion and says nothing about how the scheme behaves on that model.

Storage is tiered, and the tiers have different lifetimes. Model weights go to an ephemeral overlay volume of roughly 1 TB; experimental outputs go to a 20 GB persistent volume. Section 4 describes a failure we caused by treating those two as interchangeable.

3.2. Models and configuration

The target is Stable Diffusion XL base 1.0 on a DDIM scheduler and the attacker proxy is Stable Diffusion 2.1 base. Stability withdrew the original SD 2.1 repository while we were working, so we use a mirror of the same weights. Generation and inversion both run at 512×512 with 50 steps and guidance scale 7.5. Tree-Ring uses the released defaults: ring pattern, circular mask, radius 10, channel 3, complex injection, `l1_complex` measurement, giving $df = 634$. The SDXL threshold is $\tau = 0.0261008646426459$, taken from the original calibration at a 1% false-positive rate.

Seeds need care in interpreting our results. The base seed is 123 and trial i generates at seed $123 + i$, so the six trials use 123 through 128. The Tree-Ring key is *not* varied: `w_seed` is fixed at 999999 for every trial. All eighteen observations therefore share one watermark key, which is the study’s sharpest limitation and is discussed in Section 8.

The recorded session ran PyTorch 2.10.0+cu128 on CUDA 12.8, Python 3.12.13, transformers 4.46.3 and diffusers 0.31.0.

3.3. Running the released attack

For reference, and because it is the shortest possible statement of what the original artifact does, the attack is invoked as a single command:

```
python run_reprompting.py \
  --wm_type TR \
  --modelid_target \
    stabilityai/stable-diffusion-xl-base-1.0 \
  --modelid_attacker \
    Manojb/stable-diffusion-2-1-base \
  --resolution 512 --seed 123
```

Listing 1: The released Reprompt entry point, exactly as our notebook invokes it. Everything else in this paper is reproduction and instrumentation work built around this one command.

3.4. Provenance

All our modifications live in a fork of the released repository [9], pinned by commit hash and listed in the Availability section. We chose that over runtime patching for two reasons. Monkey-patching in the notebook process is invisible to anything launched as a subprocess, so a patched detector in the parent does not apply to the attack script it starts. And a dirty working tree satisfies a commit-hash check while running code that is not in the recorded commit. The notebook therefore issues `git reset --hard` to the pinned SHA followed by `git clean -fd`, re-reads HEAD, and asserts both that it matches and that `git status --porcelain` comes back empty.

Session metadata is serialised at the start of every run: commit hashes for our fork and the upstream base, model iden-

tifiers, dtypes, device assignment, library versions, and the SHA-256 of the resolved dependency lock. It ships with the artifacts.

3.5. Gates

We ran the reproduction as a sequence of feasibility gates rather than as an experiment. Each one asks a question that would make the rest pointless if answered badly, and none is meant to produce a finding about watermarking:

- G1** Do both pipelines fit on the hardware?
- G2** What does one attack cost in wall-clock time?
- G3** Can the detector’s underlying statistic be recovered, and does it carry usable range?
- G4** Do genuine, clean and forged observations separate?

4. Environment Reconstruction

Four things broke before anything ran. We report them as findings rather than as a narrative because each one generalises past this particular repository.

4.1. Package removal is not idempotent in layered images

The notebook image ships PyTorch 2.10 and transformers v5; the released code wants the late-2024 stack. Installing the repository’s pinned requirements on top produces a tree where v5 and v4 files sit side by side, because pip cannot fully remove distributions baked into a lower image layer. Every entry point then dies at import with a `ModuleNotFoundError` naming a module the codebase never references, which sends you looking for a missing dependency instead of a corrupted one.

Getting out requires uninstalling, deleting the package directories from `site-packages` by hand, installing five exact pins (transformers 4.46.3, tokenizers 0.20.3, diffusers 0.31.0, huggingface_hub 0.26.2, accelerate 1.1.1), and restarting the interpreter. The restart is not optional, since the half-imported broken modules stay cached in `sys.modules` and survive reinstallation.

One dependency is easy to overlook. `lpips` is in the requirements file but not among the version-critical pins, and without it every entry point fails inside the image-utility module before a model is ever loaded.

4.2. Cache paths are frozen at import

The Hugging Face hub library resolves its cache location the first time it is imported. Setting `HF_HOME` after any transitive import does nothing, so weights land in the default location, which for us was the 20 GB persistent volume they then filled. The variable has to be set in the first executed cell, above every import in the session. We point weights at the ephemeral high-capacity volume and keep the persistent one for outputs.

4.3. Guards must interrogate state, not sentinel files

Our first version of the pinning cell wrote a lock file on success and skipped reinstallation whenever that file existed. The two objects have different lifetimes: the persistent volume survives across sessions, installed packages do not. So a stale lock file

caused the pinning step to be skipped in an environment that no longer had the pins, and the session carried on with the platform’s stock `diffusers`. The guard reported success the whole time. We only caught it by reading library versions out of the session metadata.

The fixed guard calls `importlib.metadata.version` on each pinned distribution and compares against the requirement, treating `PackageNotFoundError` as failure. The general lesson is that a guard has to interrogate the state it claims to protect. A marker stored on a medium with a different lifetime than the guarded state will eventually tell you something false.

4.4. Half precision needs the autoencoder handled explicitly

The released code sets `DTYPE = torch.float32` at module level in its pipeline provider and has no half-precision path at all. We introduced `fp16` on the target to buy headroom. Detection then returned `NaN` latents while the image-quality metrics stayed well-formed, which told us generation was fine and only the detection path was broken.

The cause is specific. SDXL’s autoencoder carries `force_upcast = True` in its configuration, and the `diffusers` pipeline honours that flag inside its own `__call__`. The repository, however, calls the autoencoder directly from `vae_encode` and `latents_to_imgs`, which bypasses that handling entirely. In `fp16` those direct calls overflow. Our fork adds a `_vae_upcast` helper that saves the autoencoder’s dtype, promotes it to `fp32` when it is `fp16` and `force_upcast` is set, and restores it afterwards, wrapped around every direct encode and decode.

To check the patched path we built a probe that splits inversion into its two stages, `imgs_to_latents` (autoencoder encode) and `invert_z0` (the U-Net DDIM loop), and runs each under both dtypes against one fixed generated image so the autoencoder dtype is the only thing changing. After the patch, neither stage produced non-finite values under either configuration and both returned the same detection p -value of 1.6648×10^{-36} to the last digit.

That result validates the fix. It does not show that an unpatched `fp16` autoencoder would have been safe, and we want to be careful about the difference: the probe ran against a fork that already contains the upcast, so what it demonstrates is that the mixed-precision path preserves detector output on the tested image, not that the original overflow was imaginary. The attacker model stays in `fp32` regardless. It needs the headroom through a 50-step inversion chain and at 5.57 GB there is nothing to gain from shrinking it.

5. Detector Instrumentation

5.1. The detector discards its own statistic

The released Tree-Ring detector computes σ , λ and x exactly as in Equation 1, evaluates the non-central χ^2 CDF, and returns only the resulting p -value; its accuracy accessor returns $1 - p$. The intermediate quantities are computed and thrown away.

For a detector whose job is to accept or reject, that is a sensible interface. It gets limiting as soon as a consumer needs

to compare evidence strengths instead of outcomes. A CDF value is a tail probability, so the whole range of strong evidence gets squeezed into a narrow band next to zero where floating-point resolution starts to matter more than the underlying measurement. The accuracy accessor saturates at 1.0 and loses more still. And differences in p do not correspond to comparable differences in how well the recovered spectrum actually matches the key.

So we reimplemented Equation 1 with x , λ , df and σ all kept (Listing 2). The computation is the detector’s; only the return value differs.

```

1 def raw_tr_stat(wm, latents):
2     lat_fft = torch.fft.fftshift(
3         torch.fft.fft2(latents), dim=(-1, -2))
4     mask, gt = wm.watermarking_mask[0], wm.gt_patch[0]
5     out = []
6     for one in lat_fft:
7         f = one[mask].flatten(); t = gt[mask].flatten()
8         f = torch.concatenate([f.real, f.imag])
9         t = torch.concatenate([t.real, t.imag])
10        sigma = f.std()
11        lambd = (t ** 2 / sigma ** 2).sum().item()
12        x = (((f - t) / sigma) ** 2).sum().item()
13        out.append({"x": x, "lambd": lambd, "df":
14                    len(t),
15                    "sigma": sigma.item(),
16                    "p_value": scipy.stats.ncx2.cdf(
17                        x=x, df=len(t), nc=lambd)})
18    return out

```

Listing 2: Recovering the discarded statistic. The arithmetic mirrors the released detector; the intermediates are kept rather than collapsed into a CDF.

5.2. Validating the reimplementaion

A reimplemented statistic is only useful if it agrees with the one the detector applies. On every observation we push our recovered x back through the same CDF and compare against what the released detector returns for that latent, asserting agreement below 10^{-12} . The assertion held on all eighteen observations at a maximum absolute difference of exactly 0. Our reimplementaion is not an approximation of the detector’s computation; it is the same computation. That matters because it rules out reimplementaion error as an explanation for anything in Section 6.

5.3. The choice of score is a modelling decision

Two scores fall out of Equation 1 naturally. The raw discrepancy $-x$ is the simpler one: lower x means a closer match to the key. The adjusted form $\lambda - x$ offsets the discrepancy by the non-centrality parameter, which is itself image-dependent through σ and so varies between observations.

They are not the same thing written two ways. Across our runs σ spans 31.75 to 42.59 and λ spans 856.6 to 1541.4, so the two scores are not monotonically related. On the same eighteen observations the forged arm separates from the clean null at AUC 0.861 under $-x$ and 0.972 under $\lambda - x$; the genuine arm reaches 1.000 under both. Figure 5 shows the two curves.

A gap that size, produced purely by a choice of transformation, means the score has to be fixed in advance rather than chosen once separation is visible. Picking between candidate scores after seeing how each performs on the samples you

Algorithm 1 One measurement trial. u_T is the target, u_A the attacker proxy, $\mathcal{I}$ inversion, $\mathcal{G}$ generation, $\mathcal{D}$ the Tree-Ring detector and k the key. Prompts q_T and q_A are unrelated.

```

1:  $z_T^{(w)} \leftarrow \text{WATERMARK}(k)$  ▷ key-derived latent
2:  $x^{(g)} \leftarrow \mathcal{G}(u_T, q_T, z_T^{(w)})$  ▷ genuine
3:  $x^{(c)} \leftarrow \mathcal{G}(u_T, q_T, \varepsilon)$ ,  $\varepsilon \sim \mathcal{N}(0, I)$  ▷ clean null
4:  $\hat{z}^{(w)} \leftarrow \mathcal{I}(u_A, x^{(g)})$  ▷ invert in attacker model
5:  $x^{(f)} \leftarrow \mathcal{G}(u_A, q_A, \hat{z}^{(w)})$  ▷ forged
6: for  $x \in \{x^{(g)}, x^{(c)}, x^{(f)}\}$  do
7:      $\hat{z} \leftarrow \mathcal{I}(u_T, x)$ 
8:     record  $(x, \lambda, \text{df}, \sigma, p) \leftarrow \mathcal{D}(\hat{z}, k)$ 
9: end for

```

then report is selection on the test set. We report both and treat neither as canonical.

5.4. Measurement harness

The released attack script writes only the censored p -value to its metrics output, so our measurement runs do not go through it. We assemble the attack from the repository’s own primitives, with inversion, generation and detection all unmodified, and add a third arm the script does not produce. Each trial yields three observations against a common watermark instance:

- **genuine** — a watermarked latent generated in the target and inverted back;
- **clean** — the same prompt generated from ordinary Gaussian noise, an empirical null for both other arms;
- **forged** — the genuine image inverted in the attacker model, regenerated there under an unrelated prompt, and handed back to the target detector.

The clean arm is the addition that earns its place. Without an empirical null there is no way to say what a given score is evidence *for*; you can only say whether it crossed a threshold.

Algorithm 1 states one trial. Line 4 is the whole vulnerability: the ring pattern survives inversion through a model that never held the key, so line 5 can regenerate an unrelated image from a latent that still carries it. The two models sit on separate devices and are swapped to host memory around each other’s turn.

To be explicit about the boundary: the attack concept, the DDIM inversion, the Tree-Ring detector, the diffusion models and the prompt sets are Müller et al.’s. The loop assembly, the clean null arm, the recovery of the discarded statistic and its validation are ours.

6. Evaluation

All results below come from six trials of three arms, $n = 18$ observations, at generation seeds 123 through 128 under the single fixed Tree-Ring key.

6.1. G1 — memory feasibility

Figure 2 gives the measured peaks. The attacker at fp32 reaches 5.57 GB, the target at fp16 reaches 7.36 GB, and

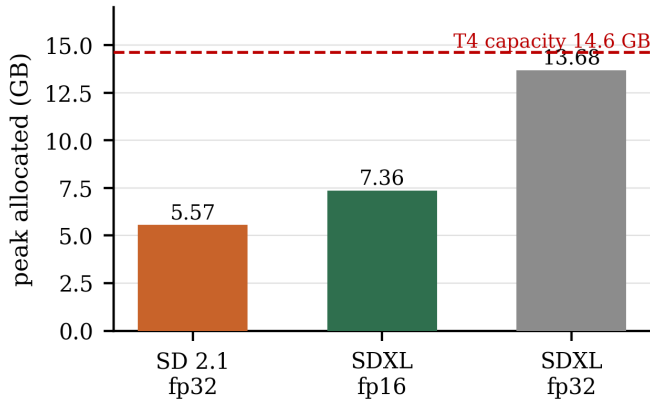


Figure 2: Peak allocated memory per pipeline, from `torch.cuda.max_memory_allocated` after one generation. Every configuration fits on a single T4 individually; half precision on the target frees 6.32 GB.

the target at fp32 reaches 13.68 GB. All three fit on a single 14.6 GB card individually, which is what matters here: our configuration places the attacker on `cuda:0` and the target on `cuda:1`, so the relevant question is per-device headroom rather than the sum. **G1 passes.**

Half precision on the target is therefore not required for Reprompt to run on this hardware. What it buys is headroom: 6.32 GB of it, leaving roughly 7.2 GB free on the target device rather than under 1 GB. We adopted it for that margin, not out of necessity.

6.2. G2 — wall-clock cost

One Reprompt run through the released entry point completes in 332.0 s on a T4. Müller et al. report roughly 30 seconds for the smaller models and up to two minutes for FLUX.1 in single-GPU, single-batch runs, with their full experimental campaign spread over eight A40s. Our figure is therefore several times their published single-GPU range, which is about what the difference in memory bandwidth and tensor-core generation would predict. **G2 passes.**

The number that matters for a threat model is not the ratio but the absolute cost. Forgery is a sub-six-minute operation on hardware anyone can access for free.

6.3. G3 — statistic recoverability and range

All eighteen observations produced distinct values of x , spanning 889.1 to 1802.2 against $\text{df} = 634$, and the cross-check against the released detector returned a maximum absolute difference of exactly 0. The statistic is recoverable and carries range. **G3 passes.**

The p -value did not behave the way we predicted. From reading the detector source we had inferred that `scipy.stats.ncx2.cdf` would underflow to exactly 0.0 on strong genuine detections, left-censoring the genuine distribution and making $-\log_{10} p$ undefined, and we had used that inference to justify recovering x in the first place. What we measured: p was exactly zero in 0 of 18 observations, all eighteen values were distinct, and the smallest was 5.45×10^{-49} . The prediction failed. The case for recovering the raw statis-

Table 2: Per-arm outcomes ($n = 6$ each) beside the values Müller et al. report for the same model pairing.

Arm	$\bar{x}$	$\bar{\lambda} - x$	detected	orig.
genuine	1096.5	+220.0	6/6	1.00
clean	1623.9	−626.3	0/6	0.01
forged	1391.2	−371.3	5/6	0.97

Table 3: Detection p -values. Means are dominated by the largest observation in each arm, so medians are given alongside.

Arm	median p	mean p	orig. mean p
genuine	8.63×10^{-38}	7.96×10^{-24}	1.10×10^{-21}
clean	4.56×10^{-1}	4.80×10^{-1}	4.72×10^{-1}
forged	2.30×10^{-5}	1.50×10^{-2}	5.20×10^{-3}

tic still stands on the grounds in Section 5.3, but not on the grounds we originally gave for it.

6.4. G4 — hypothesis separation

The attack reproduces. Tables 2 and 3 put our measurements next to the original work’s. The clean arm lines up closely, with a mean p of 0.480 against 0.472 and no false positives in six trials against a reported rate of 0.01. That is the arm that most directly validates the detector as we configured it. The genuine arm is detected every time. The forged arm crosses $\tau = 0.0261$ in five of six trials at a median p of 2.30×10^{-5} , three orders of magnitude clear of the threshold. Running the released entry point directly gave the same picture: a genuine p of 3.53×10^{-33} and a forged p of 3.13×10^{-6} , both detected.

Separation. Against the clean null the genuine arm separates perfectly, AUC 1.000 on both scores. The forged arm reaches 0.861 on $-x$ and 0.972 on $\lambda - x$. **G4 passes.**

Hypothesis tests. Because $n = 6$ per arm can make a single AUC look more conclusive than it is, we also ran a small battery of post-hoc tests over the same eighteen observations, all of which are reproduced by the accompanying `stat_analysis.py` on the published `phase0_scores.csv`. A 95% bootstrap interval on the forged-vs-clean AUC spans $[0.58, 1.00]$ for $-x$ and $[0.83, 1.00]$ for $\lambda - x$. A two-sided Mann-Whitney test separates the forged arm from the clean null at $p = 0.041$ and the genuine arm from the clean null at $p = 0.002$, with the two arms sitting 1.67 and 2.84 pooled standard deviations below the null in the raw statistic (Cohen’s d). The genuine-forged comparison is the loosest of the three at $p = 0.065$. That ordering is Figure 3’s intermediate band in different clothes: the forged arm is measurably closer to valid evidence than the null is, while still not being confusable with a genuine image on six draws. Figure 6 shows both candidate scores side by side.

One divergence worth naming. Our forged detection rate of $5/6 = 0.83$ comes in under the 0.97 reported originally. At

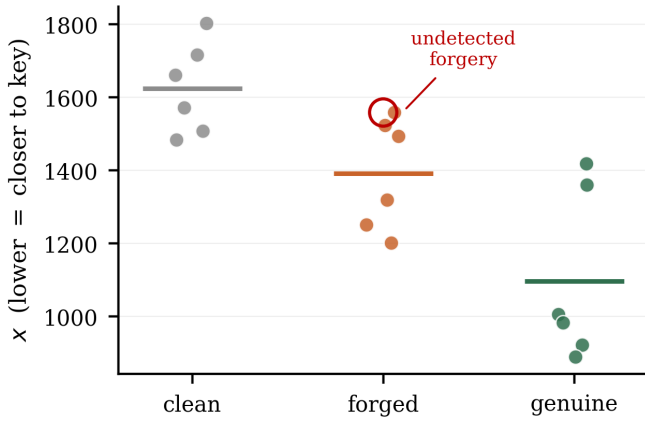


Figure 3: Recovered statistic x by arm, one point per trial, with arm means marked. The forged arm sits between the clean null and the genuine arm. The circled point is the single forgery the detector missed; its statistic falls inside the clean range.

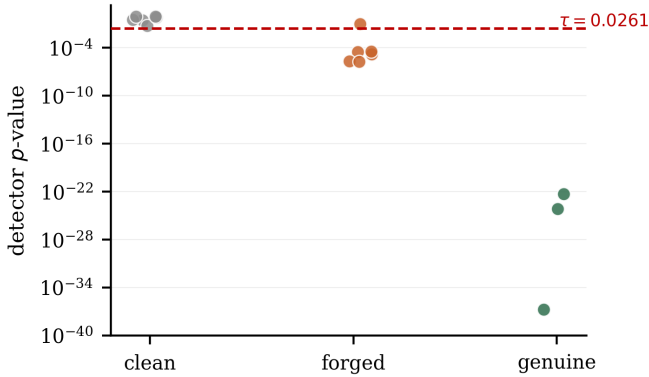


Figure 4: Detector p -values on a log scale against the calibrated threshold $\tau = 0.0261$. No observation reached exactly zero; the smallest was 5.45×10^{-49} .

$n = 6$ that difference is not meaningful; a 95% Wilson score interval [16] on 5/6 runs from 0.44 to 0.97 and contains their value. We are not claiming the attack is weaker than reported.

The shape of the failure is more interesting than the rate. Figure 3 shows the forged arm occupying a band between the clean null and the genuine arm, and the one forgery the detector missed has $x = 1558.3$, sitting inside the clean range of 1483.7 to 1802.2. It did not fail narrowly: its x value fell inside the observed clean range.

That intermediate band is the observation we would carry forward. A detector working as a binary test reports the five successful forgeries and the six genuine images identically. Whatever would distinguish them is visible in the raw statistic and gone from the thresholded decision, and the detector computes it either way.

7. Discussion

7.1. Where the effort actually went

Invoking the released attack takes one command, shown in Listing 1. Our reproduction notebook is 375 lines. Table 4 breaks that down, and the ratio is the part worth saying out loud: the published artifact is not the expensive part of repro-

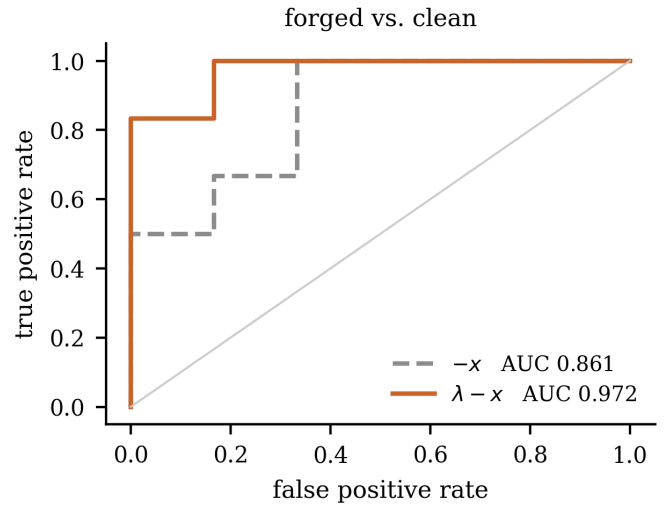


Figure 5: Separating forged from clean under the two candidate scores. Both are derived from the same eighteen observations; only the transformation differs.

Table 4: Distribution of reproduction code by purpose.

Component	Lines	Category
Environment repair, provenance	132	portability
Memory measurement (G1)	36	instrumentation
Invoking the released attack	1	original artifact
Timing wrapper (G2)	22	instrumentation
Precision patch and probe	49	portability
Statistic recovery and harness	71	extension
Analysis and gate verdicts	64	extension

ducing a published artifact.

None of this is a complaint about the artifact, which ran and produced results consistent with its paper. It is an observation about what artifact evaluation measures. Executability gets checked in the environment the authors specify. Portability across hardware generations, driver stacks and library versions is a different property, and it is the one that decides whether the work can be built on a year later by someone with different hardware. About 45% of our code exists only to construct an environment where the other 55% can run.

7.2. Detector interfaces decide what can be asked

The result we would most like to see carried elsewhere concerns the detector’s return type rather than the attack. Returning a thresholded decision or a CDF value is enough for deployment and not enough for analysis. Our whole instrumentation effort exists to undo one line of the released detector: the decision to return p instead of the intermediate detector quantities (x, λ, df, σ) that p is computed from.

Exposing those quantities costs nothing, since they are already computed. Not exposing them costs a downstream consumer a reimplementa-tion that then has to be validated against the original, which is exactly the work we did in Section 5. Detectors meant for research use should return these intermediate quantities alongside whatever decision they make. This is an argument about interface design, not about the correctness of

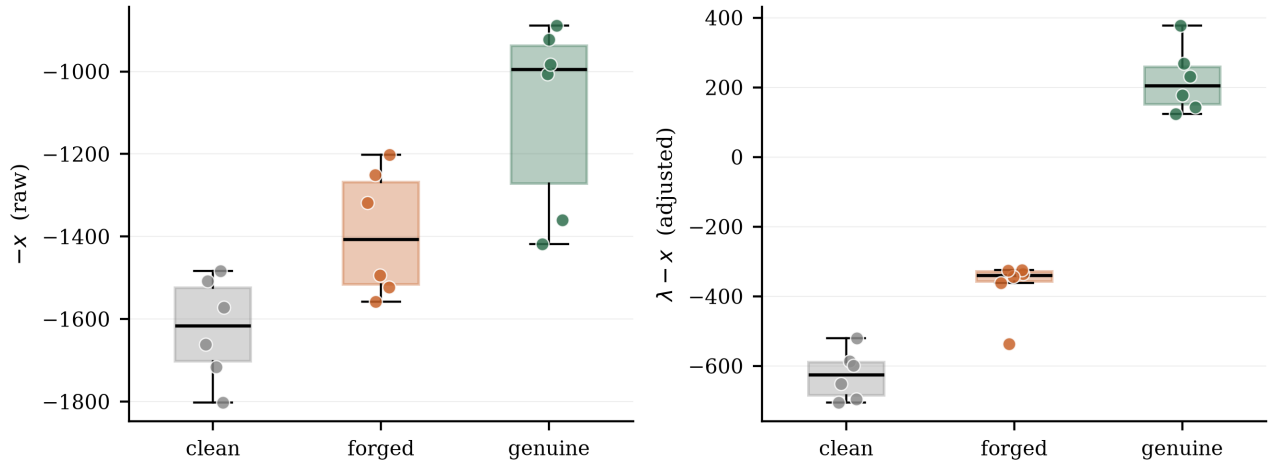


Figure 6: Per-arm distributions of both candidate scores, boxes spanning the interquartile range with the individual trials overlaid. The genuine arm clears the clean null on both scores; the forged arm occupies the gap between them.

the scheme.

7.3. Predictions that did not survive measurement

Two of our inferences came from reading code rather than running it, and both were wrong in the same way.

The first was that the detector’s p -value is left-censored at zero. The reasoning was not silly: a non-central χ^2 CDF evaluated deep into its lower tail is a plausible underflow candidate. Direct measurement disproved it. None of the eighteen p -values was zero, all eighteen were distinct, and the minimum was 5.45×10^{-49} .

The second concerned half precision. The targeted autoencoder upcast resolved the observed failure, but our controlled probe was run after the patch was already in the fork (Section 4.4). It therefore validates the patched path rather than independently localising the original fault. A probe that cannot fail is not a test, and we report the result as the former rather than the latter.

There is a third, smaller correction: we first attributed a PSNR calculation fix to commit `43eec7e`, which touches documentation only. The fix is `d5885a3`.

What links them is that in each case we reasoned from what the code appeared to do rather than from what it measurably did, and in the second case built the check only after the fix. We report them because that shortcut is available at every point in a reproduction, and because a study that only surfaces the predictions that held has not shown it could catch one that did not. Arp et al. [2] catalogue related failure modes in machine-learning security research; ours is a variant of their point that assumptions carried into an evaluation tend to survive it unexamined.

7.4. What this says about the attack

Our reproduction supports the original conclusion. Forgery worked in five of six trials on the first configuration we tried, at 332.0 s per attempt on free hardware, using a proxy that shares no weights with the target. The attacker needs one watermarked reference image, and the median forged p -value lands three orders of magnitude below the calibrated threshold.

The property being exploited is specific rather than general. Tree-Ring detection is a similarity test against a recoverable artifact, and the secret used to verify is the same secret used to inject. These two properties help explain why this attack shape applies to Tree-Ring and to related inversion-based semantic watermarking schemes, and the pattern’s transferability across models means the attacker never needs the target model itself. We reproduced only Tree-Ring, so we make no claim about schemes we did not test, and this is not a general law relating robustness to forgeability. Schemes that verify with a secret distinct from the one they inject with are not implicated, which is precisely the direction MetaSeal [19] takes with content-dependent cryptographic signatures, and which Gunn et al. [6] approach differently by selecting initial latents with a pseudorandom error-correcting code so that watermarked and unwatermarked outputs are computationally indistinguishable. What it does mean is that robustness evaluations of this family measure a property orthogonal to the one a provenance claim rests on.

8. Limitations

We state these plainly, because several of them materially constrain what our numbers support.

A single watermark key. Every observation shares one Tree-Ring seed (`w_seed = 999999`). The variation we report is within-key. These are not independent draws over the key space, and the separation in Section 6 must not be read as a distribution over watermarks. This is the study’s most serious limitation and the first thing we correct in subsequent work. It matters more here than it would for a robustness study, since Ci et al. [3] showed the multi-key setting is exactly where Tree-Ring’s behaviour departs from the single-key case.

Sample size. Eighteen observations, six per arm. Enough to establish that the attack reproduces and the statistic is recoverable; not enough for a distributional claim. The AUC values should be read as ordering indicators rather than calibrated estimates, and the gap between our 5/6 forged detection rate and the reported 0.97 cannot be resolved at this size.

Attack scope. Reprompt only. The optimisation-based Imprint variant costs roughly two orders of magnitude more per run by the original authors’ own reporting, which puts a replication with meaningful sample size outside our compute allocation. We make no claim about how Imprint behaves on this hardware.

Precision divergence. The original results are fp32 throughout; ours are mixed precision with a patched autoencoder path. Our probe found fp16 and fp32 detection identical on one image after the patch (Section 4.4). That is a narrow result, and it does not establish that fp16 is numerically neutral for the detection statistic in general.

Excluded target models. FLUX.1-dev needs `bfloat16`, unavailable on the T4. PixArt- Σ is supported by the released code but runs a DPM scheduler rather than DDIM, and the original work’s own ablation shows the scheduler substantially affects latent recovery quality, so an SDXL/PixArt comparison would confound architecture with scheduler. We report neither.

Pristine images only. Every observation is of an unprocessed image. Realistic handling such as recompression, resizing or screenshotting is untested, and both populations would need to pass through matched channels before anything could be concluded about behaviour under it.

Attacker model substitution. Stability withdrew the original SD 2.1 repository during this work and we used a mirror. We verified the weights load and produce the expected attack behaviour, but did not compare them bitwise against the original release.

9. Conclusion

We reproduced the Reprompt semantic watermark forgery attack against Tree-Ring on Stable Diffusion XL, using the authors’ released implementation on free-tier dual T4 GPUs with substantially less per-GPU memory than the original evaluation environment. The attack reproduces: genuine images detected six times out of six, clean images none, forged images five out of six, at 332.0 s per attempt. The clean arm tracks the original work closely at a mean p of 0.480 against 0.472, which is the arm that most directly validates the detector as configured.

The finding we would most want carried forward has to do with the detector’s interface rather than the attack. The released detector computes a non-central χ^2 statistic and returns only its CDF. Recovering that statistic, exactly rather than approximately, shows that two natural scores built from it separate the forged arm at AUC 0.861 and 0.972 on identical samples, which makes the choice of score something that has to be fixed before separation is observed. It also shows the forged arm occupying an intermediate band between the clean null and the genuine arm. The detector computes that information and then discards it at the moment it returns a decision.

We also report predictions that failed. We had inferred the detector’s p -value was left-censored at zero; across eighteen observations it was never zero, with a minimum of 5.45×10^{-49} . We had assumed the half-precision fault localised to the autoencoder before building anything capable of testing it,

and the probe we did build ran against an already-patched fork. In both cases an artifact of presentation was read as evidence about underlying values. A reproduction that surfaces only its confirmed predictions has not shown it could catch one that failed.

The immediate next step is to vary the watermark key across clusters, which is the binding limitation here, and to pass both genuine and forged populations through matched laundering channels so the distributions being compared have had identical handling.

Division of Work

Saifur Rahman Tamim designed the gate protocol, built the environment reconstruction and provenance pinning, wrote the mixed-precision patch and probe, and implemented the statistic recovery and its validation. Md Taslimul Hasan Toufique carried out the post-hoc statistical analysis of the gate data (bootstrap confidence intervals, pairwise Mann-Whitney tests and effect sizes reported in Section 6), produced the distributional figure (Fig. 6) and the script that regenerates it, drafted the related-work section, and ran the notebook sessions alongside the first author. A.M. Tayeful Islam advised on experimental design and reviewed the manuscript. All authors contributed to the analysis and the writing.

Availability

Two repositories accompany this work. The reproduction notebook, the gate artifacts (`gate1.json`, `gate3.json`, `gate4.json`, `precision_probe.json`, `phase0_scores.csv`, `phase0_verdict.json`, `session_meta.json`), the statistical re-analysis script (`stat_analysis.py`), the figure-generating script and the source of this paper are at <https://github.com/sr-tamim/watermark-forgery-research>.

The modified attack code is a fork of Müller et al.’s released repository [9], at <https://github.com/sr-tamim/semantic-forgery>, pinned for all runs reported here at commit `7f9e7ad`, which branches from upstream `ca68950`. Both hashes are recorded in `session_meta.json` alongside the SHA-256 of the resolved dependency lock, so any run can be traced to the exact code that produced it.

References

- [1] Bang An, Mucong Ding, Tahseen Rabbani, Aakriti Agrawal, Yuancheng Xu, Chenghao Deng, Sicheng Zhu, Abdirisak Mohamed, Yuxin Wen, Tom Goldstein, and Furong Huang. Benchmarking the robustness of image watermarks. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.08573.
- [2] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don’ts of machine learning in computer security. In *USENIX Security Symposium*, 2022.

- [3] Hai Ci, Pei Yang, Yiren Song, and Mike Zheng Shou. RingID: Rethinking tree-ring watermarking for enhanced multi-key identification. In *European Conference on Computer Vision (ECCV)*, pages 338–354, 2024.
- [4] Ingemar J. Cox, Matthew L. Miller, Jeffrey A. Bloom, Jessica Fridrich, and Ton Kalker. *Digital Watermarking and Steganography*. Morgan Kaufmann, 2nd edition, 2007.
- [5] Pierre Fernandez, Guillaume Couairon, Hervé Jégou, Matthijs Douze, and Teddy Furon. The stable signature: Rooting watermarks in latent diffusion models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 22466–22477, 2023.
- [6] Sam Gunn, Xuandong Zhao, and Dawn Song. An undetectable watermark for generative image models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.07369.
- [7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [8] Dongjun Hwang, Sungwon Woo, Tom Gao, Raymond Luo, and Sunghwan Baek. Invisible watermarks: Attacks and robustness. *arXiv preprint arXiv:2412.12511*, 2024.
- [9] Andreas Müller. semantic-forgery. <https://github.com/and-mill/semantic-forgery>, 2024. Accessed August 2026; pinned at commit ca68950.
- [10] Andreas Müller, Denis Lukovnikov, Jonas Thietke, Asja Fischer, and Erwin Quiring. Black-box forgery attacks on semantic watermarks for diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2412.03283.
- [11] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukyab, Patrick Traynor, and Kevin Butler. “get in researchers; we’re measuring reproducibility”: A reproducibility study of machine learning papers in tier 1 security conferences. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.
- [12] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. SDXL: Improving latent diffusion models for high-resolution image synthesis. In *International Conference on Learning Representations (ICLR)*, 2024.
- [13] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [14] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.
- [15] Yuxin Wen, John Kirchenbauer, Jonas Geiping, and Tom Goldstein. Tree-rings watermarks: Invisible fingerprints for diffusion images. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [17] Zijin Yang, Kai Zeng, Kejiang Chen, Han Fang, Weiming Zhang, and Nenghai Yu. Gaussian shading: Provable performance-lossless image watermarking for diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12162–12171, 2024.
- [18] Xuandong Zhao, Kexun Zhang, Zihao Su, Saastha Vasan, Ilya Grishchenko, Christopher Kruegel, Giovanni Vigna, Yu-Xiang Wang, and Lei Li. Invisible image watermarks are provably removable using generative AI. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [19] Tong Zhou, Ruyi Ding, Gaowen Liu, Charles Fleming, Ramana Rao Kompella, Yunsu Fei, Xiaolin Xu, and Shaolei Ren. MetaSeal: Defending against image attribution forgery through content-dependent cryptographic watermarks. *Transactions on Machine Learning Research*, 2026. arXiv:2509.10766.
- [20] Chaoyi Zhu, Zaitang Li, Renyi Yang, Robert Birke, Pin-Yu Chen, Tsung-Yi Ho, and Lydia Y. Chen. Optimization-free universal watermark forgery with regenerative diffusion models. *arXiv preprint arXiv:2506.06018*, 2025.
- [21] Jiren Zhu, Russell Kaplan, Justin Johnson, and Li Fei-Fei. HiDDeN: Hiding data with deep networks. In *European Conference on Computer Vision (ECCV)*, pages 657–672, 2018.